

Module 2

Combinational Logic Circuits

2.1 Combinational Logic representation

Combinational circuits

- Digital circuits are divided into two broad categories:
 - Combinational circuits
 - Sequential circuits
1. In combinational circuits, outputs at any instant of time depend upon the inputs present at that instant of time. So no memory in these circuits
 2. In sequential circuits, outputs at any instant of time depend upon the present inputs as well as past inputs/outputs
So memory elements are required to store past information

Design requirements

- Design requirements of combinational circuits:
 - A set of statements
 - Boolean expression
 - Truth table
- In circuit design aims should be:
 - Use of minimum number of components to ensure low cost
 - Saving in space
 - Power requirements

Methods to simplify Boolean functions

- Algebraic method
- Karnaugh-map technique
- Quine-McCluskey method
- Variable entered mapping (VEM) technique
- K-map- simplest and most commonly used manual method. Convenient upto 6 variables. Beyond that it is very cumbersome
- Quine-McClusky- Suitable for computer mechanisation

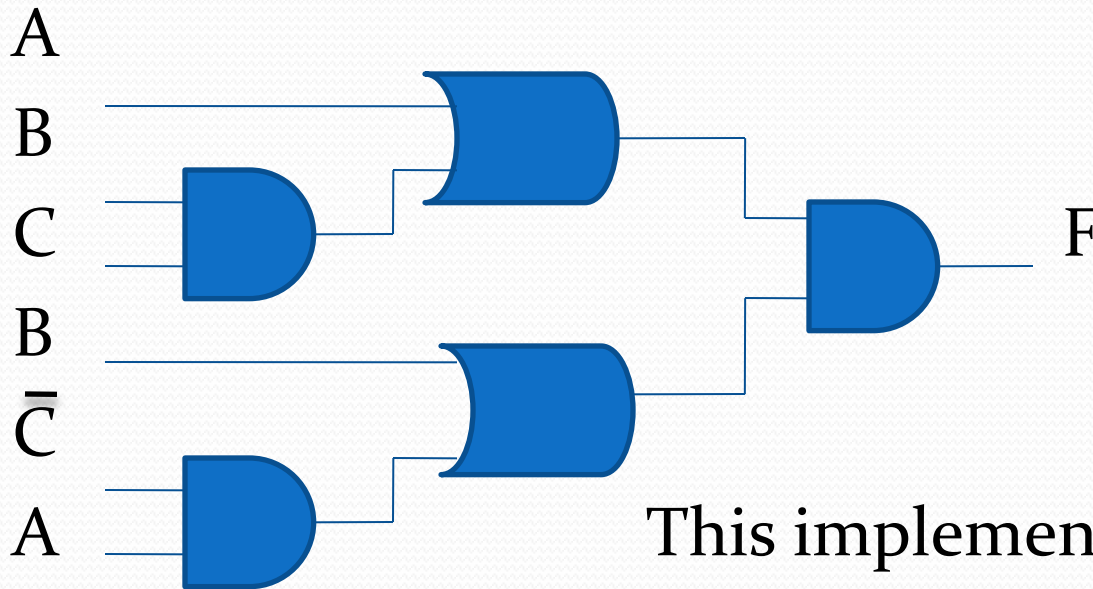
Standard representations for logic functions

1. Sum-of-products form (SOP)
2. Product-of-sums form (POS)

e.g. given logic expression is

$$F = (A+BC)(B+\overline{C}A) \rightarrow 1$$

This function can be implemented using basic gates as :



This implementation requires 3 levels

Sum-of-product form (SOP)

$$\begin{aligned} Y &= (A+BC)(B+\bar{C}\bar{A}) \text{----} \rightarrow 1 \\ &= A(B+\bar{C}\bar{A}) + BC(B+\bar{C}\bar{A}) \text{--} \rightarrow 2 \\ &= AB + A\bar{C}\bar{A} + BCB + BC\bar{C}\bar{A} \text{--} \rightarrow 3 \end{aligned}$$

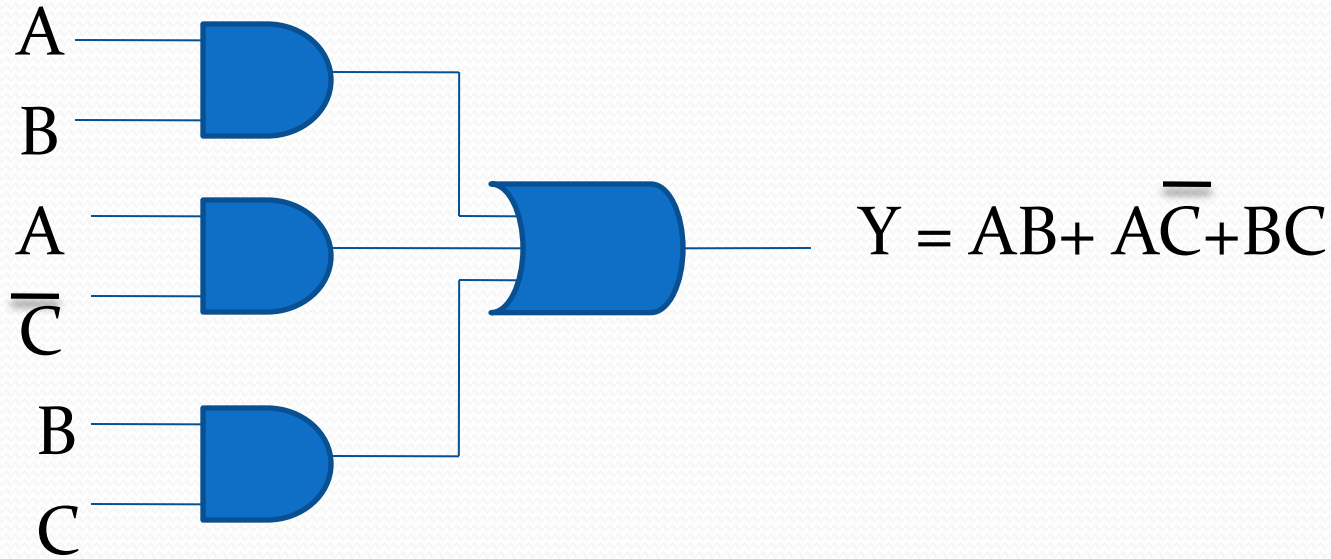
Eqn 3 can be simplified using Boolean laws as follows:

$$= AB + A\bar{C} + BC \text{--} \rightarrow 4$$

Representation of eqn 4 is known as sum-of-products(SOP) form. This can be realized with AND-OR gates configuration.

SOP realization using AND-OR gates

$$Y = AB + A\bar{C} + BC$$



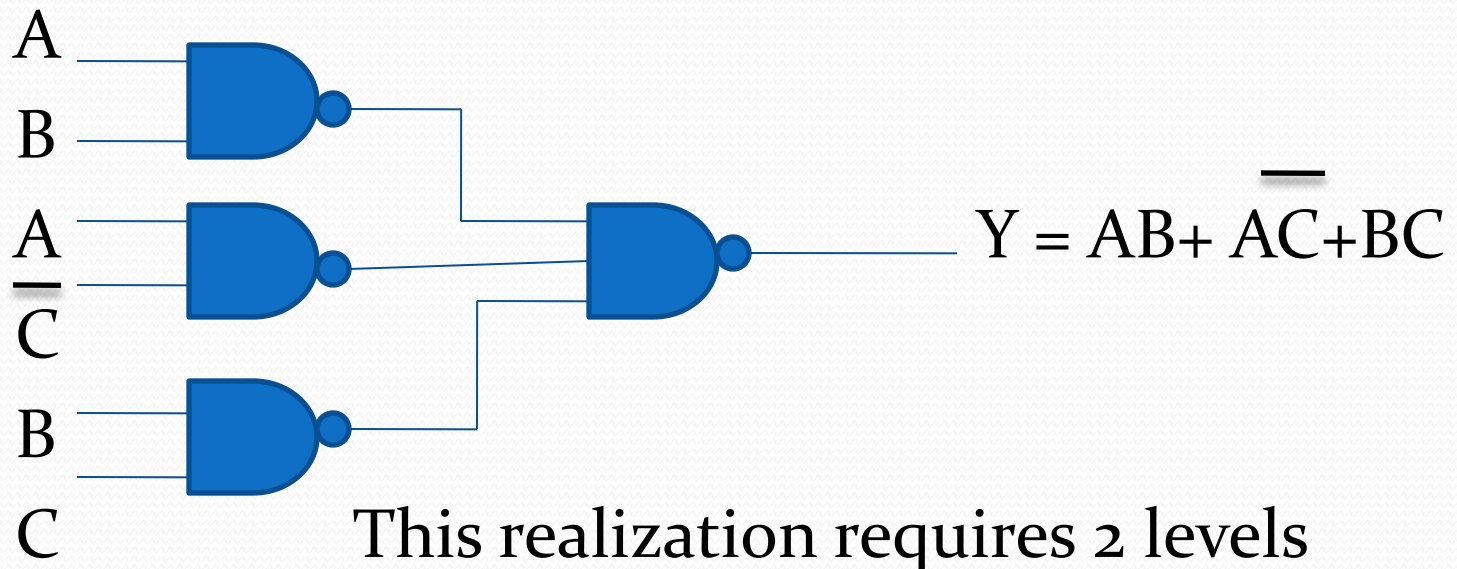
This realization requires 2 levels

SOP realization using NAND gates

- Using De-Morgan's theorems eqn $AB + AC' + BC$ can be written as

$$\begin{aligned}\overline{Y} &= \overline{(AB + AC' + BC)} \rightarrow 1 \\ &= AB \cdot \overline{AC'} \cdot \overline{BC} \rightarrow 2\end{aligned}$$

Eqn 2 can be realized using NAND gates only as follows:



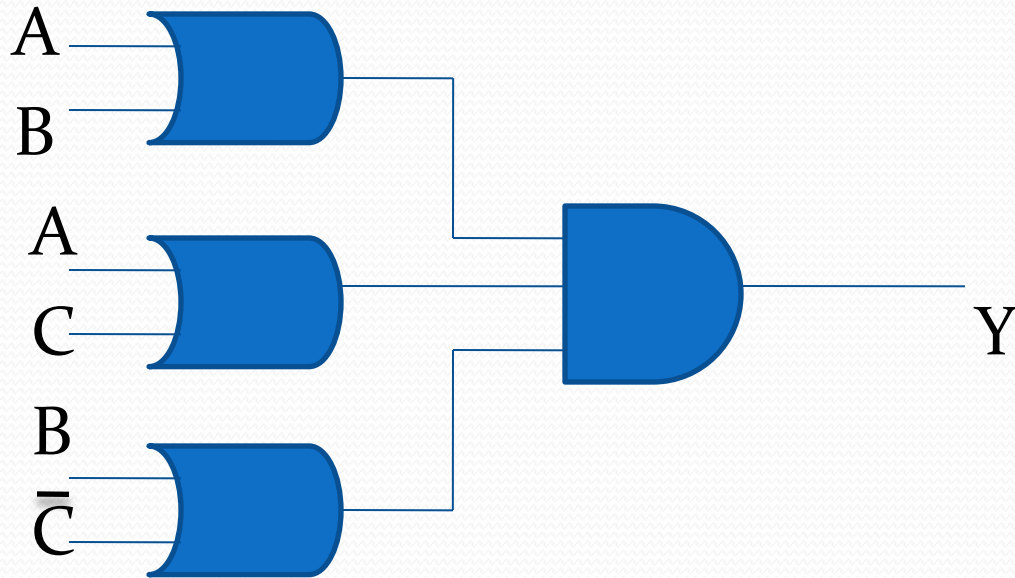
Product-of-sum form (POS)

$$Y = (A+BC)(B+\bar{C}A) \rightarrow 1$$

- Eqn 1 can be represented in another form using Boolean laws as
- $Y = (A+B)(A+C)(B+\bar{C})(B+A) \rightarrow 2$
- Eqn 2 can be simplified using Boolean laws as follows:
- $Y = (A+B)(A+C)(B+\bar{C}) \rightarrow 3$
- Eqn 3 is known as Product-of-sums (POS) form

POS realization using OR-AND

$$Y = (A+B)(A+C)(B+\bar{C})$$



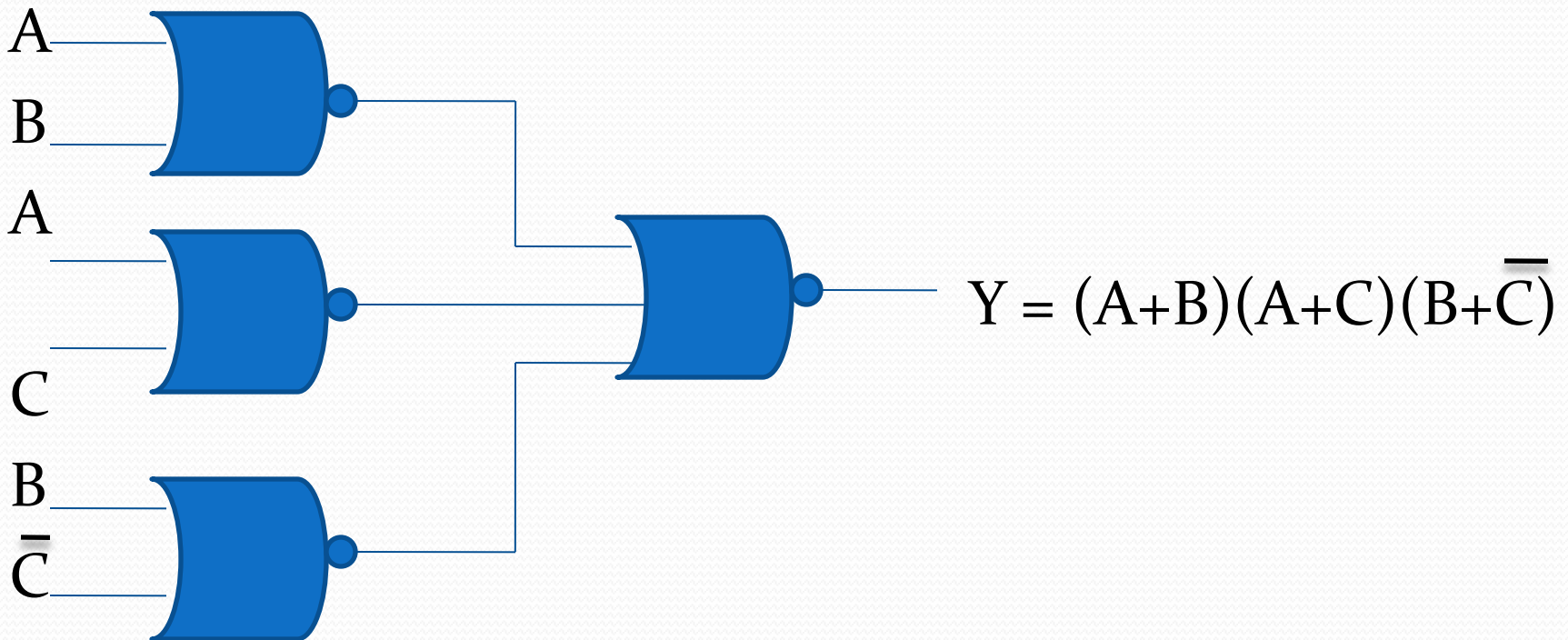
POS realization using NOR gates

Using De-Morgan's theorems eqn $Y = (A+B)(A+C)(B+\overline{C})$

Can be written as $\overline{Y} = \overline{(A+B)(A+C)(B+\overline{C})} \rightarrow 2$

$$= \overline{(A+B)} + \overline{(A+C)} + \overline{(B+\overline{C})} \rightarrow 3$$

Eqn 3 can be realized using NOR gates only

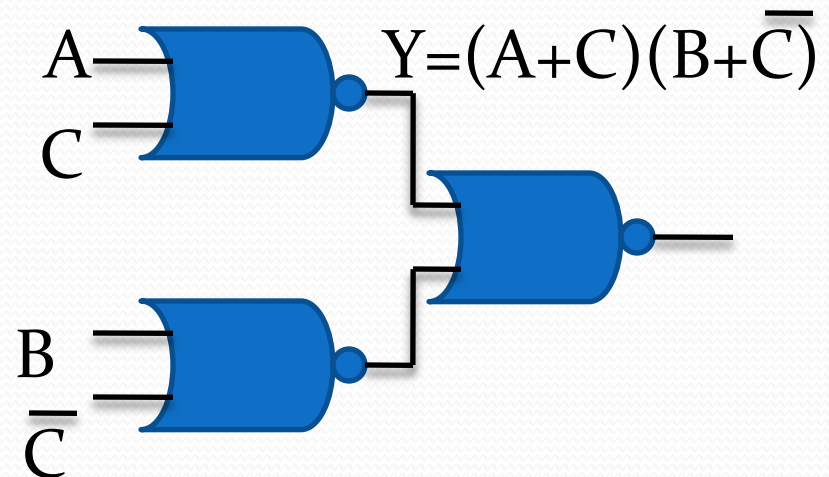
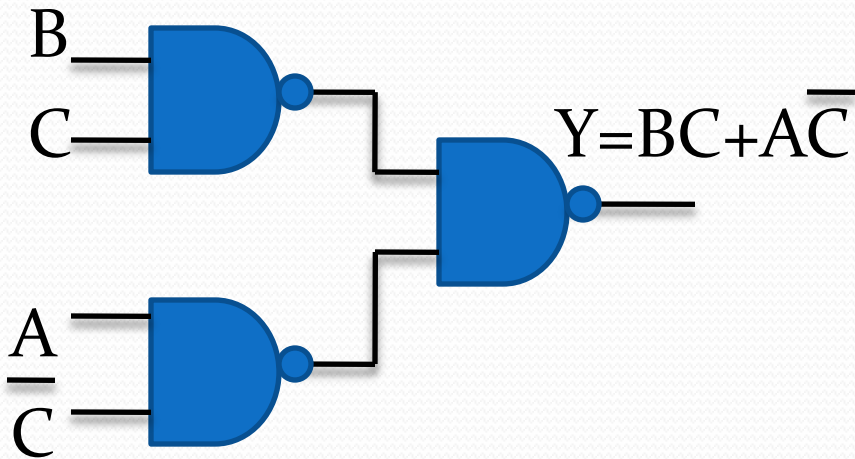


Simplification of Boolean equations

1) Eqn $Y = AB + \bar{A}C + BC$ can be simplified using Boolean law as $Y = BC + A\bar{C}$ --- $\rightarrow_1$

2) Eqn $Y = (A+B)(A+C)(B+\bar{C})$ can be simplified using Boolean law as $Y = (A+C)(B+\bar{C})$ --- $\rightarrow_2$

Realization of equations 1 and 2



Min terms and Max terms

- Eqns $Y = AB + A\bar{C} + BC$ -----(SOP form)
and $Y = (A+B)(A+C)(B+\bar{C})$ --- (POS form)
- Each individual input variable is termed as *Literal*.
- If each term in SOP and POS forms contains all the literals then these are known as *canonical* SOP and POS respectively
- Each individual term in canonical SOP form is called as *minterm*
- Each individual term in canonical POS form is called as *maxterm*

Conversion of SOP form in canonical SOP

- SOP form can be converted into canonical SOP by ANDing the terms in the expression with terms formed by ORing the variable and its complement which are not present in that term
- E.g for 3 variable expression with variables A, B and C, if there is term A only where B and C terms are missing, then we form two terms $(B + \bar{B})$ and $(C + \bar{C})$ and AND with A
- So we get $A(B + \bar{B}).(C + \bar{C}) = ABC + A\bar{B}C + AB\bar{C} + A\bar{B}\bar{C}$

Example

- Convert SOP eqn $Y = AB + A\bar{C} + BC$ into canonical SOP form
- $Y = AB(C + \bar{C}) + A\bar{C}(B + \bar{B}) + BC(A + \bar{A})$
- $= ABC + AB\bar{C} + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}BC + \bar{A}\bar{B}C$
- $= ABC + AB\bar{C} + A\bar{B}\bar{C} + \bar{A}\bar{B}C$

Conversion of POS form into canonical POS form

- ORing the terms in the expression with terms formed by ANDing the variable and its complement which are not present in that term
- E.g. for 3 variable expression with variables A, B and C if there is a term A where B and C are missing then we form BB and CC and OR A with these terms. So we get
- $A + B\bar{B} + C\bar{C} = (A + B\bar{B} + C)(A + B\bar{B} + \bar{C})$
- $= (A + B + C)(A + \bar{B} + C)(A + B + \bar{C})(\bar{A} + B + \bar{C})$

Canonical SOP representation in min terms

Canonical POS representation in max terms

- $Y = ABC + AB\bar{C} + A\bar{B}\bar{C} + \bar{A}BC$

This canonical SOP eqn can be represented as min terms:

- $Y = m_3 + m_4 + m_6 + m_7$
 $= \sum m(3,4,6,7)$

- $Y = (A+B+C)(A+\bar{B}+\bar{C})(\bar{A}+B+\bar{C})$

This canonical POS eqn can be represented as max terms:

$$Y = M_0 \cdot M_1 \cdot M_2 \cdot M_5$$

$$= \prod M(0,1,2,5)$$

Variables			Min terms	Max terms
A	B	C	m_i	M_i
0	0	0	$A' B' C' = m 0$	$A + B + C = M 0$
0	0	1	$A' B' C = m 1$	$A + B + C' = M 1$
0	1	0	$A' B C' = m 2$	$A + B' + C = M 2$
0	1	1	$A' B C = m 3$	$A + B' + C' = M 3$
1	0	0	$A B' C' = m 4$	$A' + B + C = M 4$
1	0	1	$A B' C = m 5$	$A' + B + C' = M 5$
1	1	0	$A B C' = m 6$	$A' + B' + C = M 6$
1	1	1	$A B C = m 7$	$A' + B' + C' = M 7$

Karnaugh map(K-map) representation of logic functions

- Graphical method of simplifying and manipulating Boolean expressions or truth table in systematic way
- Information available in truth table or available in POS or SOP form is represented on Karnaugh map (K-map)
- The map is made up of squares, each square representing one min term of the function
- This produces a circuit diagram with minimum number of gates and minimum number of inputs
- Most extensively used tool
- Can be used for any number of variables but generally used up to 6 variables beyond which it becomes very cumbersome

Y \ X	0	1
0		
1		

YZ \ X	0	1
00		
01		
11		
10		

Z \ XY	00	01	11	10
0				
1				

YZ \ WX	00	01	11	10
00				
01				
11				
10				

Two variable K-map

		X	
		0	1
Y	0	0	2
	1	1	3

2-Variable K-Map

		X	
		0	1
Y	0	00	10
	1	01	11

		X	
		0	1
Y	0	$X'Y'$	XY'
	1	$X'Y$	XY

Note:

- Logically adjacent cells are physically adjacent in the k-map
- Each cells has two adjacent cells

		X	
		0	1
Y	0	m0	m2
	1	m1	m3

Two variable k-map with SOP and POS mapping

A. SOP: -

		B	
		$\bar{B}$ 0	B 1
A	$\bar{A}$ 0	$\bar{A}.\bar{B}$	$\bar{A}.B$
	A 1	$A.\bar{B}$	$A.B$

B. POS: -

		B	
		B 0	$\bar{B}$ 1
A	A 0	$A+B$	$A+\bar{B}$
	$\bar{A}$ 1	$\bar{A}+B$	$\bar{A}+\bar{B}$

Example of mapping two variable truth table into k-map

2 Variable Truth Table

	A	B	F
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1



		K-Map	
		0	1
B	A		
	0	0 0	1 2
1	1	1 1	1 3

Three variable K-map

XY		X			
		00	01	11	10
Z	0	0	2	6	4
	1	1	3	7	5

Y

Z

A. SOP: -

A\BC		$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
		00	01	11	10
$\bar{A}$	0	$\bar{A}\bar{B}\bar{C}$ 0	$\bar{A}\bar{B}C$ 1	$\bar{A}BC$ 3	$\bar{A}B\bar{C}$ 2
A	1	$A\bar{B}\bar{C}$ 4	$A\bar{B}C$ 5	ABC 7	$AB\bar{C}$ 6

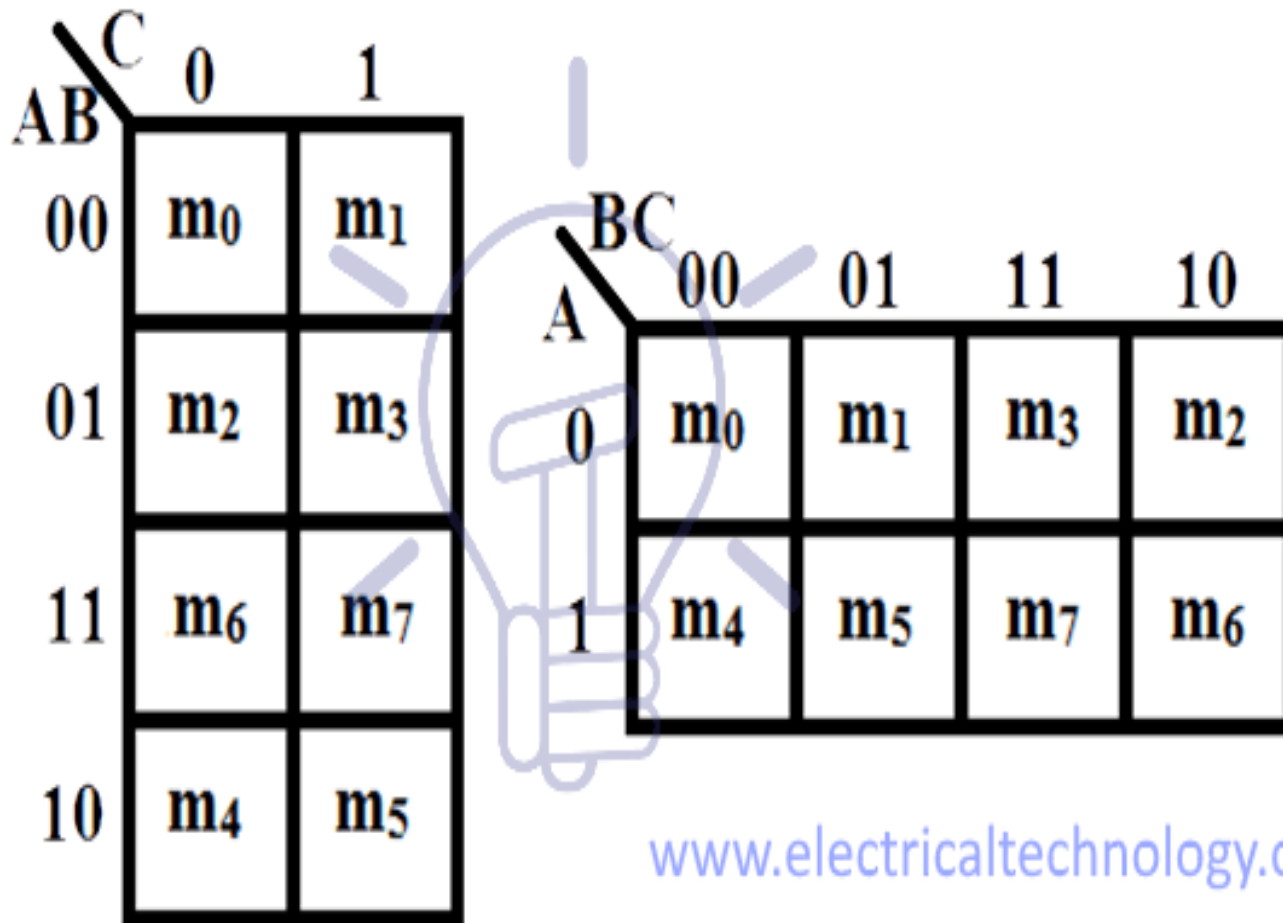
AB\C		$\bar{C}$	C
		0	1
$\bar{A}\bar{B}$	00	$\bar{A}\bar{B}\bar{C}$ 0	$\bar{A}\bar{B}C$ 1
$\bar{A}B$	01	$\bar{A}B\bar{C}$ 2	$\bar{A}BC$ 3
AB	11	$AB\bar{C}$ 6	ABC 7
$A\bar{B}$	10	$A\bar{B}\bar{C}$ 4	$A\bar{B}C$ 5

B. POS: -

A\B+C		$B+C$	$B+\bar{C}$	$\bar{B}+\bar{C}$	$\bar{B}+C$
		00	01	11	10
A	0	$A+B+C$ 0	$A+B+\bar{C}$ 1	$A+\bar{B}+\bar{C}$ 3	$A+\bar{B}+C$ 2
$\bar{A}$	1	$\bar{A}+B+C$ 4	$\bar{A}+B+\bar{C}$ 5	$\bar{A}+\bar{B}+\bar{C}$ 7	$\bar{A}+\bar{B}+C$ 6

A+B\C		C	$\bar{C}$
		0	1
$A+B$	00	$A+B+C$ 0	$A+B+\bar{C}$ 1
$A+\bar{B}$	01	$A+\bar{B}+C$ 2	$A+\bar{B}+\bar{C}$ 3
$\bar{A}+\bar{B}$	11	$\bar{A}+\bar{B}+C$ 6	$\bar{A}+\bar{B}+\bar{C}$ 7
$\bar{A}+B$	10	$\bar{A}+B+C$ 4	$\bar{A}+B+\bar{C}$ 5

3 variable k-map with min term locations



3 variable k-map example 1

A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

=

A B		C	
		0	1
00		1	0
01		1	0
11		1	0
10		1	0

3 variable k-map example 2

$A B C$	F
0 0 0	0
0 0 1	0
0 1 0	1
0 1 1	1
1 0 0	1
1 0 1	0
1 1 0	1
1 1 1	0

(a)

$A \backslash BC$		0	1
		0	1
00	0	0	1
01	0	0	0
11	1	0	0
10	1	1	1

(b)

$ABC = 001, F = 0$

$ABC = 110, F = 1$

Four variable K-map

AB

		AB			
		00	01	11	10
CD	00	0	4	12	8
	01	1	5	13	9
	11	3	7	15	11
	10	2	6	14	10

ABCD

0000 - 0

0001 - 1

0010 - 2

0011 - 3

0100 - 4

0101 - 5

0110 - 6

0111 - 7

ABCD

1000 - 8

1001 - 9

1010 - 10

1011 - 11

1100 - 12

1101 - 13

1110 - 14

1111 - 15

$\begin{array}{c} ab \\ \hline cd \end{array}$	$a'b'$ 00	$a'b$ 01	ab 11	ab' 10
$c'd'$ 00	0 (0000)	4 (0100)	12 (1100)	8 (1000)
$c'd$ 01	1 (0001)	5 (0101)	13 (1101)	9 (1001)
cd 11	3 (0011)	7 (0111)	15 (1111)	11 (1011)
cd' 10	2 (0010)	6 (0110)	14 (1110)	10 (1010)

Fig. 2.20(a) 4-variable K-map (Format 1)

$\begin{array}{c} cd \\ \hline ab \end{array}$	$c'd'$ 00	$c'd$ 01	cd 11	cd' 10
$a'b'$ 00	0	1	3	2
$a'b$ 01	4	5	7	6
ab 11	12	13	15	14
ab' 10	8	9	11	10

Fig. 2.20(b) 4-variable K-map (Format 2)

CD		00	01	11	10
AB	00	⁰ $A'B'C'D'$	¹ $A'B'C'D$	³ $A'B'CD$	² $A'B'CD'$
	01	⁴ $A'BC'D'$	⁵ $A'BC'D$	⁷ $A'BCD$	⁶ $A'BCD'$
	11	¹² $ABC'D'$	¹³ $ABC'D$	¹⁵ $ABCD$	¹⁴ $ABCD'$
	10	⁸ $AB'C'D'$	⁹ $AB'C'D$	¹¹ $AB'CD$	¹⁰ $AB'CD'$

AB

		AB			
		00	01	11	10
CD	00	0	4	12	8
	01	1	5	13	9
	11	3	7	15	11
	10	2	6	14	10

ABCD

0000 - 0

0001 - 1

0010 - 2

0011 - 3

0100 - 4

0101 - 5

0110 - 6

0111 - 7

ABCD

1000 - 8

1001 - 9

1010 - 10

1011 - 11

1100 - 12

1101 - 13

1110 - 14

1111 - 15

K map with don't care conditions

Don't-care condition

- ❑ Minterms that may produce either 0 or 1 for the function.
- ❑ Marked with an 'x' in the K-map.
- ❑ These don't-care conditions can be used to provide further simplification.

		A'		B'		B	
		BC		00	01	11	10
A'	0	1	x	x	1		
A	1	0	x	0	1		
				C'	C	C'	BC'

Five variable structure

$\bar{A}$					A						
$\overline{DE}$		BC				$\overline{DE}$		BC			
		00	01	11	10			00	01	11	10
$\overline{BC}$	00	m_0	m_1	m_3	m_2	$\overline{BC}$	00	m_{16}	m_{17}	m_{19}	m_{18}
	01	m_4	m_5	m_7	m_6		01	m_{20}	m_{21}	m_{23}	m_{22}
	11	m_{12}	m_{13}	m_{15}	m_{14}		11	m_{28}	m_{29}	m_{31}	m_{30}
	10	m_8	m_9	m_{11}	m_{10}		10	m_{24}	m_{25}	m_{27}	m_{26}

K-Map with “Don’t Care” Conditions (1)

Sometimes there occurs situation in which input variable combinations are not allowed (e.g. BCD invalid) but you still need to do minimization. The “not allowed input variable combinations” = don’t care can be used to the advantage of K-Map. If SOP use the ‘X’ as 1; if POS use the ‘X’ as 0.

Example :

Input Output

A	B	C	z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	x
1	0	0	x
1	0	1	1
1	1	0	1
1	1	1	1

(a)

} “don’t
care”

	$\bar{C}$	C
$\bar{A}\bar{B}$	0	0
$\bar{A}B$	0	x
AB	1	1
$A\bar{B}$	x	1

(b)



	$\bar{C}$	C
$\bar{A}\bar{B}$	0	0
$\bar{A}B$	0	0
AB	1	1
$A\bar{B}$	1	1

(c)

z = A

3 variables with output “don’t care (X)”

		CD			
		00	01	11	10
AB	00		1	X	1
	01		X	1	1
	11	X	1	1	1
	10	1			